Anomaly Detection & Application Reliability with Endpoint Analytics

Implementation Solution Guide

Version 1.0.0 · 2026-07-21

Contents

Executive Summary — Anomaly Detection & Application Reliability with Endpoint Analytics 4

1. Solution Overview — Anomaly Detection & Application Reliability with Endpoint Analytics 4

What this solution is . . . 4

How it works . . . 5

The three signals, and how to read them . . . 6

The triage-and-close-the-loop practice . . . 6

Where this sits relative to the other Endpoint Analytics products . . . 6

Benefits . . . 7

Limitations (be honest with buyers) . . . 7

The standard deployment model . . . 8

2. Requirements — Anomaly Detection & Application Reliability with Endpoint Analytics 8

R1. Endpoint Analytics enabled and reliability data flowing (see the parent product) . . . 8

R2. Licensing / entitlement for Endpoint Analytics . . . 9

R3. Anomaly detection available in your tenant/cloud . . . 9

R4. Sufficient device population reporting reliability data . . . 9

R5. Device scope and data-collection scope defined . . . 9

R6. Reader access to Endpoint Analytics reports . . . 9

R7. Permission to create the review-scope group (state-changing step only) . . . 10

R8. The parent and automated-fix companion solutions available (recommended) . . . 10

Global Requirements — Intune Solutions (category-level) 10

G1. Microsoft Entra ID tenant . . . 10

G2. Microsoft Intune subscription . . . 10

G3. Administrative roles . . . 10

G4. Device enrollment in Intune . . . 11

G5. Pilot security group . . . 11

G6. Supported, current operating system . . . 11

G7. Network access to required cloud endpoints . . . 11

3. Implementation — Anomaly Detection & Application Reliability with Endpoint Analytics 11

Phase 1 — Confirm reliability data is flowing (Assess) . . . 12

Phase 2 — Read application reliability & separate bad-app from bad-rollout (Configure) . . . 12

Phase 3 — Read device reliability & restart analytics (Configure) . . . 13

Phase 4 — Use anomaly detection to catch a regression early (Pilot) . . . 14

Phase 5 — Triage, route, and close the loop (Validate) . . . 14

Phase 6 — Establish the review cadence and expand (Expand) . . . 15

5. Testing & Validation — Anomaly Detection & Application Reliability with Endpoint Analytics 16

Test 1 — Reliability signals are populated . . . 16
Test 2 — Pilot readiness gate . . . 17
Test 3 — A known bad app is correctly identified . . . 17
Test 4 — Bad app vs. bad rollout is separated correctly . . . 17
Test 5 — A restart trend is correctly attributed . . . 17
Test 6 — An anomaly is correctly interpreted (true signal) . . . 17
Test 7 — A false signal is correctly rejected . . . 17
Test 8 — Triage routes to the right owner and closes the loop . . . 18
Test 9 — The review cadence and export work end to end . . . 18
Acceptance gate . . . 18
6. Troubleshooting — Anomaly Detection & Application Reliability with Endpoint Analytics 18
Reliability data is missing or delayed . . . 18
The anomaly view is empty or unavailable . . . 19
False anomalies (a signal that isn't real) . . . 19
Can't attribute a crash to a specific app version . . . 19
Restart cause is unclear . . . 20
The fix shipped but the signal didn't recover . . . 20
Licensing & scope . . . 20
Scripts . . . 21
7. Best Practices — Anomaly Detection & Application Reliability with Endpoint Analytics 21
Read the signals the right way . . . 21
Separate a bad app from a bad rollout — every time . . . 22
Route to the right owner and close the loop . . . 22
Hand scriptable fixes to the automated-fix companion . . . 22
Lean on the parent for the data, not this product . . . 22
Make it a cadence, and measure it . . . 23
Respect national-cloud differences . . . 23
8. Deliverables Summary — Anomaly Detection & Application Reliability with Endpoint Analytics 23
Documentation (this guide) . . . 23
PowerShell scripts (Microsoft Graph) — `ENDPTREL-Scripts.zip` . . . 24
Hero image . . . 24
How it's delivered . . . 24
Appendix A — PowerShell Scripts 25

Executive Summary — Anomaly Detection & Application Reliability with Endpoint Analytics

Reliability problems hide in aggregate. One line-of-business app crashes on a few hundred devices; an OS update quietly raises restart rates; a driver regresses on a single hardware model. None of it shows up as a single dramatic outage — it shows up as a **slow rise in tickets** that nobody connects to a cause until it's widespread. Microsoft Intune's **Endpoint Analytics** already surfaces these through **application reliability**, **device reliability / restart analytics**, and **anomaly detection** — but the signals only help if someone knows how to **read them and act**.

This solution operationalizes the reliability side of Endpoint Analytics:

- **Application reliability** → read crashes and hangs **by app and version**, and **separate a bad app from a bad rollout** so you fix the right thing.
- **Device reliability & restart analytics** → turn "random reboots" into an attributable cause — an OS update fleet-wide, a driver on one hardware model.
- **Anomaly detection** → catch a fleet-wide regression **early**, read what the anomaly is telling you, and rule out false signals before spending effort.
- **Triage and close the loop** → route each confirmed signal to the responsible app / driver / update owner, and hand scriptable fixes to the automated-fix companion — then verify the signal recovers.

It is deliberately scoped to the reliability + anomaly deep-dive and the triage practice that connects it to its two neighbors: the **flagship Endpoint Analytics** solution as the **parent** (which enables Endpoint Analytics and gets devices reporting) and **"Proactive Remediations at Scale"** as the **automated-fix companion** (which ships the scriptable fix at scale). This product does not reproduce either — it reads the signals and routes the work.

Built the standard repeatable way — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — with exact, click-by-click Endpoint Analytics steps (navigation flagged to verify against current Microsoft documentation, because Microsoft changes it), five ready-to-run PowerShell scripts (Microsoft Graph; read-only by default, with a single `-WhatIf`-gated state-changer), and testing/troubleshooting guides with clear pass/fail gates.

Outcome: a reliability practice built on Endpoint Analytics that catches app crashes, restart spikes, and fleet-wide anomalies early — and routes each to the right fix **before** it becomes a ticket wave — run by your own team, without a consulting engagement.

1. Solution Overview — Anomaly Detection & Application Reliability with Endpoint Analytics

What this solution is

This solution operationalizes the **reliability side of Microsoft Intune's Endpoint Analytics**. Where the flagship **"Endpoint Analytics Deployment & Optimization in Microsoft Intune"** solution (referred to

throughout as the **parent** product) covers turning Endpoint Analytics on, getting devices reporting, and reading the startup-performance and productivity scores, **this** product goes deep on three reliability signals and — most importantly — on **what to do with them**:

1. **Application reliability** — crashes and hangs broken down **by application and version**, so you

can see which app (and which build of it) is failing on your fleet and how many users it affects.

1. **Device reliability & restart analytics** — restart causes and **unexpected-restart trends**, so a

spike in reboots stops being a mystery and becomes an attributable event (an OS update, a driver, a bad configuration).

1. **Anomaly detection** — an early-warning signal that flags when a metric **regresses across a slice

of the fleet** (a device model, an app version, an OS build), so a bad app rollout or a driver regression is caught while it's small instead of after it becomes a wave of tickets.

The heart of the product is not the reports themselves — it's the **triage-and-close-the-loop practice** wrapped around them: how to read each signal, how to **separate a bad app from a bad rollout**, how to route the problem to the responsible **app / driver / update owner**, and how to hand a scriptable fix to the **"Proactive Remediations at Scale"** solution (referred to as the **automated- fix companion**) so the loop actually closes.

> ***Feature, licensing, and navigation accuracy.*** *Endpoint Analytics reliability reports and anomaly detection evolve, and some capabilities are in preview or vary by cloud. This guide describes the* ***concepts and the practice*** *with confidence, but flags every* ***exact blade name, navigation path, licensing requirement, and feature-availability claim*** *as* ***"verify against current Microsoft documentation"*** *— because Microsoft changes those. Never treat a flagged specific as settled without checking your own tenant.*

How it works

1. **Endpoint Analytics is already collecting data.** Enabling Endpoint Analytics, deploying the data-

collection policy, and getting devices to report are covered by the **parent** product — this solution **assumes that groundwork is done** and starts from "the reliability data is flowing."

1. **Application reliability aggregates crash/hang telemetry by app + version.** Each managed app gets

a reliability signal built from how often it crashes or hangs and how many devices/users that affects. A version dimension lets you see a specific build regress.

1. **Device reliability / restart analytics aggregates restart events.** Restarts are classified

(planned vs. unexpected / stop-error) and trended over time so an abnormal rise is visible.

1. **Anomaly detection watches for regressions.** Rather than making you eyeball every chart, anomaly

detection surfaces when a metric moves abnormally for a **cohort** — a hardware model, an app version, an OS build — which is exactly the shape of a bad rollout or a driver regression.

1. **You triage and route.** For each signal you decide: is this a **bad app** (fix or roll back the

app/version) or a **bad rollout / environmental** cause (an update or driver hitting a device model)? Then you route it to the owner and, where the fix is scriptable, hand it to the **automated-fix companion**.

1. **A review cadence keeps it honest.** A short, repeatable **reliability review** turns these signals

into a habit — the difference between "we have the data" and "we catch regressions early."

The three signals, and how to read them

Application reliability (bad app vs. bad rollout)

• **Bad app:** a single application (or a single **version** of it) shows elevated crashes/hangs across

a broad, mixed population of devices — many models, many OS builds. The common factor is the *app*. → Fix, update, or roll back the **app/version**; route to the **app owner**.

• **Bad rollout:** crashes/hangs spike **right after** a new app version or configuration was deployed,

concentrated in the population that received it. The common factor is the *change event*. → Pause/stage the rollout, roll back the version, route to the **deployment owner**.

Device reliability & restart analytics

• **Unexpected-restart trend rising** across the fleet, or across a **single device model**, usually

means an **update or driver** rather than an app. A model-scoped spike points at a **driver/firmware** regression on that hardware; a fleet-wide spike right after an OS update points at the **update**. → Route to the **hardware/driver owner** or the **update owner**; hand the mitigation (roll back a driver, block an update ring, apply a config) to the **automated-fix companion** where scriptable.

Anomaly detection (catch the regression early)

• An anomaly is the system telling you **"this metric just moved abnormally for this cohort."** Read

which metric (crashes, restarts, a specific reliability measure), **which cohort** (model / app version / OS build), and **when it started** — the start time almost always lines up with a change.

• **True signal vs. false signal:** a true anomaly correlates with a real change event and persists; a

false signal is transient (a one-off, a reporting gap, a tiny cohort with high variance) and fades on the next data refresh. The practice below shows how to tell them apart before you spend effort.

The triage-and-close-the-loop practice

This is the repeatable model the whole solution is built around:

> ***Read the signal → classify (bad app vs. bad rollout vs. environmental) → confirm it's real (not a false anomaly) → route to the responsible owner → hand a scriptable fix to the automated-fix companion → verify the signal recovers → record it.***

Every step in the Implementation guide serves this loop, and the review cadence runs it on a schedule.

Where this sits relative to the other Endpoint Analytics products

• **Parent — "Endpoint Analytics Deployment & Optimization in Microsoft Intune":** turns Endpoint

Analytics on, deploys data collection, and covers startup performance and the productivity/score side. **Do that first.** This solution does **not** re-teach enabling Endpoint Analytics — it points you back to the parent for the groundwork and picks up at reading reliability data.

• **Automated-fix companion — "Proactive Remediations at Scale — Detection & Remediation Script Packs

for Microsoft Intune": **where a reliability problem has a** scriptable **fix (clear a bad cache, reset a service, remove a bad driver, repair a config), this solution** hands the fix off** to the companion's detection/remediation script model rather than reproducing it here.

This product is deliberately scoped to the **reliability + anomaly deep-dive** and the triage practice that connects the signals to those two neighbors.

Benefits

• **Catch regressions early** — anomaly detection flags a bad rollout, a driver regression, or an update raising restarts **while it's small**, before it becomes a ticket wave.

• **Separate a bad app from a bad rollout** — a clear, repeatable classification so you fix the *right* thing instead of guessing.

• **Attributable restarts** — restart analytics turns "random reboots" into an owner and a cause.

• **Right fix, right owner** — every signal routes to the app/driver/update owner and, where scriptable, straight into the automated-fix companion.

• **A durable practice, not a one-off** — the review cadence makes reliability a standing habit, and the practice is reusable on every future rollout.

Limitations (be honest with buyers)

• **Endpoint Analytics must already be enabled and collecting data** — this solution starts from data flowing; enabling it is the **parent** product's job.

• **Reliability data is aggregated and delayed** — signals populate over a period (often measured in days) and need a minimum device population before they're meaningful; brand-new tenants or tiny cohorts will look empty or noisy. *(Exact data-latency and minimum-device thresholds vary — verify against current Microsoft documentation.)*

• **Coverage is platform- and OS-dependent** — application reliability and restart analytics apply to the supported Windows client population; exact OS build and device-eligibility requirements **must be verified against current Microsoft documentation.**

• **Anomaly detection is an evolving capability** — availability, exact behavior, and whether it is in preview or GA **vary and must be verified against current Microsoft documentation**; treat its output as a lead to confirm, not a verdict.

• **This product interprets and routes — it does not deploy the fix.** The scriptable remediation lives in the automated-fix companion; the OS/driver/app change lives with its owner.

• **National-cloud differences** — Government (GCC / GCC High / DoD) portal URLs and feature parity can

differ; where a step names a portal or a feature, **verify the Government equivalent against Microsoft's US-sovereign-cloud documentation.**

The standard deployment model

This solution uses the same repeatable model as the rest of the catalog so repeat buyers recognize the pattern:

> ***Confirm reliability data is flowing → stand up a named review scope → read application reliability and separate bad-app from bad-rollout → read device reliability / restart analytics → use anomaly detection to catch a regression → triage and route (hand scriptable fixes to the automated-fix companion) → establish a review cadence — piloted on a small scope first, then expanded.***

Every step follows the repeatable pattern: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

2. Requirements — Anomaly Detection & Application Reliability with Endpoint Analytics

Confirm **every** requirement below before you begin. Each entry follows the same pattern: **Description · Why required · How to verify · How to implement if missing.** Category-wide prerequisites (tenant, Intune licensing, admin roles, enrollment, supported OS, network) are covered once in the **Global Requirements — Intune Solutions** document; the items here are specific to **reading Endpoint Analytics reliability signals and anomaly detection and acting on them**.

> ***Verify feature and licensing specifics against Microsoft.*** *Endpoint Analytics reliability reports, anomaly detection, licensing entitlements, minimum device populations, data latency, and the exact OS builds that report each signal* ***change over time and can differ by cloud****. Every requirement below that names a specific portal path, licensing SKU, threshold, or feature-availability claim is flagged* ***"verify against current Microsoft documentation"*** *— do that for your own tenant rather than trusting a snapshot.*

R1. Endpoint Analytics enabled and reliability data flowing (see the parent product)

Description: Endpoint Analytics is **already turned on**, the data-collection policy is deployed, and enrolled devices are reporting — specifically the **reliability** data (application reliability and device/restart analytics), not only startup performance. **Why:** This solution reads and acts on reliability signals; it does **not** enable Endpoint Analytics. Enabling it, deploying data collection, and onboarding devices are the job of the **parent** solution, *"Endpoint Analytics Deployment & Optimization in Microsoft Intune."* **Verify:** In the Intune admin center (`https://intune.microsoft.com`), open **Endpoint analytics** and confirm the reliability reports show data (not "collecting"/empty). *(Exact report location and names — verify against current Microsoft documentation.)* Run `ENDPTREL-Discovery.ps1` for a scripted readiness read. **Implement:** Deploy the **parent** solution first (enable Endpoint Analytics + the data-collection policy + onboard devices), then return here once reliability data is populating.

R2. Licensing / entitlement for Endpoint Analytics

Description: The licensing/entitlement that includes Endpoint Analytics for your managed devices. **Why:** Endpoint Analytics (and its reliability reports) are gated by entitlement; without it the reports are unavailable regardless of enrollment. **Verify:** Endpoint Analytics loads and reports render for your managed population. *(Exact licensing requirements and which SKUs include Endpoint Analytics and anomaly detection — **verify against current Microsoft documentation**; do not assume from this guide.)* **Implement:** Assign the licensing your tenant requires per Microsoft's current documentation; the parent solution covers this at the tenant level.

R3. Anomaly detection available in your tenant/cloud

Description: The **anomaly detection** capability of Endpoint Analytics is present and active for your tenant. **Why:** The anomaly-detection portion of this practice depends on the feature being available; it may be in preview, may vary by cloud (Commercial vs. GCC / GCC High / DoD), and may require a minimum device population. **Verify:** The anomaly view/report appears in Endpoint Analytics and shows either anomalies or "no anomalies." *(Whether anomaly detection is GA or preview, its exact location, and its minimum- population/latency requirements — **verify against current Microsoft documentation** for your cloud.)* **Implement:** If it is unavailable, proceed with the application- and device-reliability portions of this solution (which do not require it) and enable/adopt anomaly detection when it is available for your tenant; flag it as a follow-up.

R4. Sufficient device population reporting reliability data

Description: Enough managed devices are reporting reliability telemetry for the signals (and any anomaly cohorts) to be statistically meaningful. **Why:** Reliability scores and anomalies are **aggregate** signals; too few devices, or a brand-new tenant, produces empty or noisy output and false anomalies. **Verify:** The reliability reports show a device count consistent with your fleet, not a handful of devices. *(Minimum device counts and per-cohort thresholds — **verify against current Microsoft documentation.**)* **Implement:** Broaden the Endpoint Analytics data-collection scope (via the parent solution) and allow the reporting window to fill before drawing conclusions.

R5. Device scope and data-collection scope defined

Description: A clear understanding of **which devices** are in scope for the reliability practice (and therefore appear in the reports), driven by the Endpoint Analytics data-collection assignment. **Why:** You triage what you can see; a signal you act on must map to a real, in-scope population, and the review scope group in this solution must align with what Endpoint Analytics is collecting. **Verify:** The Endpoint Analytics data-collection policy's assignment matches the population you intend to review; the pilot group `Reliability – Pilot` (Implementation §2) is a subset of it. **Implement:** Align the data-collection assignment (parent solution) with your intended scope, then create `Reliability – Pilot` and later `Reliability – Review Scope` as subsets/aligned groups (Implementation §2 and §8).

R6. Reader access to Endpoint Analytics reports

Description: The operator has a role that can **read** Endpoint Analytics reports and device data. **Why:** Reading application reliability, restart analytics, and anomalies requires appropriate Intune/ Endpoint

Analytics read permissions. **Verify:** Entra → **Roles and administrators**; the operator holds a role that grants Endpoint Analytics read access (e.g. an Intune reader/administrator role). *(Exact role names and the least- privilege role for Endpoint Analytics — **verify against current Microsoft documentation.**)* **Implement:** Assign the appropriate read role; reserve higher privilege for the group-creating step (R7).

R7. Permission to create the review-scope group (state-changing step only)

Description: Rights to create and assign an Entra security group for the review cadence. **Why:** The one state-changing script in this solution (`ENDPTREL-Set-ReviewScope.ps1`) creates/assigns the `Reliability – Review Scope` group; that requires group-write permission (`Group.ReadWrite.All`). **Verify:** The operator can create a security group in Entra → **Groups**. **Implement:** Grant the operator group-creation rights (or have an identity admin run the state- changer). All other scripts in this solution are **read-only**.

R8. The parent and automated-fix companion solutions available (recommended)

Description: Access to the **parent** Endpoint Analytics solution and the **automated-fix companion** (Proactive Remediations at Scale). **Why:** This solution **points to** the parent for enablement and **hands scriptable fixes to** the companion; the loop closes fastest when both are on hand. Neither is technically required to *read* the signals, but the practice assumes them. **Verify:** You have the parent solution deployed (R1) and the companion available for building detection/remediation script packs. **Implement:** Deploy the parent first; adopt the companion for the remediation side. This product does **not** reproduce either — it references them.

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level on the marketplace so individual solution listings stay small. Confirm these are in place before deploying *any* Intune solution.

Each entry: **Description · Why required · How to verify · How to implement if missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, enrollment, and security policies. **Verify:** `https://intune.microsoft.com` loads and **Tenant administration** → **Tenant status** shows an active subscription. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies, enrollment connectors, and configuration profiles requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator** (or Global Administrator for connector setup). **Implement:** Assign **Intune Administrator** to the operator; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted devices are enrolled and manageable in Intune. **Why:** Policies only reach enrolled (or app-managed) devices, regardless of platform. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Configure the appropriate enrollment method for the platform and ownership model, then enroll the device.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted to a small group before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists with intended members. **Implement:** Entra → **Groups** → **New group** → type **Security** → add pilot members.

G6. Supported, current operating system

Description: A supported OS baseline with current updates for the targeted platform. **Why:** Modern Intune features and security settings assume a patched, vendor-supported OS. **Verify:** Check the device's OS version or Intune → device → **Hardware**. **Implement:** Deploy update policies or upgrade to a supported release.

G7. Network access to required cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation — Anomaly Detection & Application Reliability with Endpoint Analytics

Follow these steps **in order**. The *why* lives in the Solution Overview; the steps below are short and exact so you can **click to implement**. This solution is about **reading the reliability signals and acting on them** — it assumes Endpoint Analytics is already enabled and collecting data (the **parent** solution, *"Endpoint Analytics Deployment & Optimization in Microsoft Intune,"* does that). Pilot first: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

Consoles used:

- **Microsoft Intune admin center** — `https://intune.microsoft.com` (Endpoint analytics lives here)

• **Microsoft Entra admin center** — `https://entra.microsoft.com`

Navigation accuracy. *Endpoint Analytics report names, blade locations, tabs, and the exact place anomaly detection appears* ***change over time and can differ by cloud (Commercial vs. GCC / GCC High / DoD)****. Wherever this guide names a portal path or a report, treat the *path* as* ***"verify against current Microsoft documentation"*** *and confirm it in your own tenant — the* ***practice*** *(what to read and what to do) is what matters and does not change.*

Object names — create them exactly as written and reuse them verbatim *in every later step, the Testing guide, the Troubleshooting guide, and the scripts: - Entra security group* `Reliability – Pilot` *— the small pilot scope for this practice. - Entra security group* `Reliability – Review Scope` *— the ongoing review population (created by* `ENDPTREL-Set-ReviewScope.ps1` *in the Expand phase). - Exported report file convention* `Reliability Review – <YYYY-MM>.csv` *— the monthly reliability export produced by* `ENDPTREL-Report.ps1`*. - Handoff label convention* `Reliability Handoff – <App|Driver|Update> – <Version>` *— the ticket/ work-item label used when routing a fix to an owner or to the automated-fix companion.*

Phase 1 — Confirm reliability data is flowing (Assess)

Step 1. Confirm Endpoint Analytics reliability data is populating

1. In the Intune admin center, open **Endpoint analytics**. *(Exact left-nav location — verify against current Microsoft documentation.)*

1. Confirm the **Application reliability** report and the **device reliability / restart** report show data and a device count consistent with your fleet — not "still collecting," empty, or a handful of devices.

1. If the reports are empty or thin, **stop** and finish the **parent** solution first (enable Endpoint Analytics, deploy the data-collection policy, onboard devices) and allow the reporting window to fill. *(Data latency and minimum device population — verify against current Microsoft documentation.)*

1. For a scripted readiness read, run `ENDPTREL-Discovery.ps1` (read-only): it reports reliability-data availability, the scoped device count, and the top unreliable apps/devices it can see.

Step 2. Create the pilot review scope

1. Entra → **Groups** → **New group** → **Security**; **name it** `Reliability – Pilot`.

2. Add a small, representative set of in-scope devices (a cross-section of hardware models and app versions so the pilot exercises real triage). *(This group is a* **subset** *of what Endpoint Analytics is collecting — see Requirements R5.)*

1. You will use `Reliability – Pilot` to rehearse the read-triage-route loop before running it against the whole fleet.

Phase 2 — Read application reliability & separate bad-app from bad-rollout (Configure)

Step 3. Read the application reliability report

1. In **Endpoint analytics**, open **Application reliability**. *(Exact name/location — verify against current Microsoft documentation.)*

1. Review the apps ranked by their reliability signal (crashes/hangs and the number of devices/users affected). Note the **worst few apps** and, where the report exposes it, the **version** dimension.

1. Record the candidates you'll triage — app name, version if shown, affected device/user count — using the export from `ENDPTREL-Report.ps1` (writes `Reliability Review – <YYYY-MM>.csv`).

Step 4. Classify each candidate — bad app vs. bad rollout

For each app flagged in Step 3, decide which of these it is (Solution Overview has the full logic):

1. **Bad app** — elevated crashes/hangs spread across a **broad, mixed** population (many models, many OS builds); the common factor is the app itself. → Plan to **fix/update/roll back the app/version**; the owner is the **app owner**.

1. **Bad rollout** — the spike is **concentrated** in the population that recently received a new app **version** or configuration, and started **right after** that change. The common factor is the change event. → Plan to **pause/stage/roll back the deployment**; the owner is the **deployment owner**.

1. Confirm the split by checking the **version** breakdown: if one version is bad and the previous version is fine, it's a **bad version/rollout**; if all versions crash across the board, it's a **bad app**.

1. Run `ENDPTREL-Test-Signal.ps1` (read-only) against a known-bad app to confirm the signal is **visible and attributable** to the app/version before you act on it.

Phase 3 — Read device reliability & restart analytics (Configure)

Step 5. Read the device reliability / restart report

1. In **Endpoint analytics**, open the **device reliability / restart** report. *(Exact report name — many tenants show this as a restart-frequency / device-reliability view; verify against current Microsoft documentation.)*

1. Review restart **causes** (planned vs. **unexpected** / stop-error) and the **unexpected-restart trend** over time.

1. Look at the trend two ways:

• **Fleet-wide** rise → suspect a broadly deployed change (an **OS update**).

• **Model-scoped** rise (one hardware model spiking while others are flat) → suspect a **driver/firmware** regression on that model.

Step 6. Attribute a restart spike to an update or a driver

1. Note **when** the unexpected-restart trend rose and **which cohort** it hit (whole fleet vs. a

model).

1. Line that start time up against your change calendar — an **OS update ring**, a **driver/firmware** push, or a **configuration** change deployed around then.

1. Decide the owner: fleet-wide-after-update → **update owner**; single-model → **hardware/driver owner**.

1. Where the mitigation is **scriptable** (roll back a driver, apply a registry/config fix, pause an update ring), plan to hand it to the **automated-fix companion** (Step 9).

Phase 4 — Use anomaly detection to catch a regression early (Pilot)

Step 7. Read anomaly detection

1. In **Endpoint analytics**, open the **anomaly detection** view. *(Availability, exact name, and whether it's preview or GA — **verify against current Microsoft documentation** for your cloud; if it isn't available, skip to Step 8 and rely on the reliability trends.)*

1. For each anomaly, read the three things that make it actionable:

• **Which metric** moved (crashes, hangs, unexpected restarts, a reliability measure).

• **Which cohort** it affects (a device model, an app version, an OS build).

• **When it started** — the start time is your correlation anchor to a change event.

1. **Confirm it's real, not a false signal:**

• **True anomaly:** correlates with a real change (a rollout, an update, a driver push), affects a meaningful cohort, and **persists** across the next data refresh. → Triage it.

• **False signal:** transient, tiny/high-variance cohort, or coincides with a reporting gap, and **fades** on the next refresh. → Note it and move on; do not spend remediation effort.

1. Cross-check the anomaly against the underlying report (Step 3 or Step 5) — an anomaly should be visible as a real movement in application reliability or restart analytics, not only as an alert.

Phase 5 — Triage, route, and close the loop (Validate)

Step 8. Route each confirmed signal to its owner

1. For each confirmed bad-app / bad-rollout / restart-cause / anomaly, open (or update) a work item and label it `Reliability Handoff – <App|Driver|Update> – <Version>` (e.g. `Reliability Handoff – Contoso LOB App – 4.2.1`, or `Reliability Handoff – Display Driver – 31.0.15`).

1. Route to the owner you identified: **app owner**, **deployment owner**, **update owner**, or **hardware/driver owner**.

1. Include the evidence: the report screenshot/export row (`Reliability Review – <YYYY-MM>.csv`), the

affected cohort, and the start time.

Step 9. Hand scriptable fixes to the automated-fix companion

1. Where the fix is **scriptable and fleet-wide** — clear a bad cache, reset a service, remove/roll back

a bad driver, repair a configuration — hand it to the **"Proactive Remediations at Scale"** solution (the **automated-fix companion**) rather than fixing devices by hand.

1. Give the companion the specifics the handoff item captured: the exact condition to **detect** (which

app/version, which driver, which config) and the **remediation** to apply. *(Building the detection/remediation script pack itself is the companion's job — this solution defines **what** to fix and **who** owns it; it does **not** reproduce the companion's remediation model.)*

1. Target the companion's remediation at the affected cohort (align it with `Reliability – Review

Scope`, or a tighter model/version group where appropriate).

Step 10. Verify the signal recovers

1. After the owner ships the fix (app update/rollback, driver rollback, update-ring change, or the

companion's remediation), watch the **same** report and any related anomaly across the next data refreshes.

1. **Close the loop** only when the signal recovers: crashes/hangs fall back to baseline, the

unexpected-restart trend flattens, or the anomaly clears. Record the outcome on the handoff item.

1. Re-run `ENDPTREL-Test-Signal.ps1` against the previously-bad app/version to confirm the signal

is no longer attributable at problem levels.

Phase 6 — Establish the review cadence and expand (Expand)

Step 11. Create the ongoing review scope group

1. Run `ENDPTREL-Set-ReviewScope.ps1` — the one state-changing script — to create/assign the Entra

security group `Reliability – Review Scope` for the ongoing cadence. **It defaults to `-WhatIf` (preview only) and prints that it is doing so; re-run with `-Commit` (or `-WhatIf:$false`) to actually create/assign the group.**

1. Align `Reliability – Review Scope` with the population Endpoint Analytics is collecting (Requirements

R5) so the cadence reviews exactly what's reporting.

Step 12. Stand up the reliability review cadence

1. Establish a recurring **reliability review** (weekly or biweekly to start; monthly once steady).

Book it as a standing meeting/checklist named `Reliability Review – Weekly` (rename to match your chosen frequency).

1. Each review runs the loop against `Reliability – Review Scope`:

• Read **Application reliability** (Step 3–4) — any new bad app/version?

• Read **device reliability / restart** (Step 5–6) — any restart trend rising?

• Read **anomaly detection** (Step 7) — any new confirmed anomaly?

• **Route** new confirmed signals (Step 8), **hand scriptable fixes** to the companion (Step 9), and **verify** last cycle's fixes recovered (Step 10).

• Export the run with `ENDPTREL-Report.ps1` → `Reliability Review – <YYYY-MM>.csv` and keep it as the trend record.

1. Track **corrections-per-review over time** — a downward trend means the practice is preventing regressions instead of chasing them.

Step 13. Expand from pilot to fleet

1. Once the loop is smooth on `Reliability – Pilot`, widen the cadence to `Reliability – Review Scope` (the fleet population).

1. Validate each widening with `ENDPTREL-Validate.ps1` (reliability signals present for the scope, anomaly detection active) before relying on the review at the new scope.

1. Keep the parent solution (data collection) and the automated-fix companion (remediation) in the loop as the two neighbors this practice depends on.

5. Testing & Validation — Anomaly Detection & Application Reliability with Endpoint Analytics

Validate the reliability practice against a **pilot scope** (`Reliability – Pilot`) before you widen it to the fleet. Each test has an explicit **pass/fail gate**. Use the included PowerShell scripts (Microsoft Graph) for the scripted checks and the Endpoint Analytics reports for the read-and-interpret tests.

> ***Navigation and feature specifics — verify against current Microsoft documentation.*** *Report names, blade paths, and anomaly-detection availability change and can differ by cloud. The *tests* below are about the practice (is the signal present, correctly identified, correctly interpreted); confirm the exact portal locations in your own tenant.*

Test 1 — Reliability signals are populated

Procedure: Run `ENDPTREL-Discovery.ps1`; open **Endpoint analytics** → **Application reliability** and the **device reliability / restart** report. **Pass:** Application reliability and device/restart signals return data with a device count consistent with your scope (not empty, not a handful of devices). `ENDPTREL-Discovery.ps1` reports data present. **Fail:** Signals empty or thin → finish the **parent** Endpoint Analytics solution (enable + data collection) and allow the reporting window to fill before continuing. *(Data latency/minimum population — verify against current Microsoft documentation.)*

Test 2 — Pilot readiness gate

Procedure: Run `ENDPTREL-Validate.ps1 -PilotGroupName 'Reliability – Pilot'`. **Pass:** Application-reliability data present, device/restart data present, the `Reliability – Pilot` group exists, and anomaly detection responds (anomalies **or** an explicit "no anomalies"). Overall verdict READY. **Fail:** Any hard check FAIL → resolve it. An anomaly-detection **WARN** (unavailable/preview) is acceptable — proceed with the application- and device-reliability portions.

Test 3 — A known bad app is correctly identified

Procedure: Pick an app you know is failing (a seeded/known bad app, or the worst app in the report) and run `ENDPTREL-Test-Signal.ps1 -AppNameLike '<app>'`. Cross-check against **Application reliability**. **Pass:** The signal is **DETECTED** and attributable to that app; the crash/hang counts and affected- device count match what the report shows. **Fail:** NOT DETECTED for an app you know is failing → confirm the app is in the reporting scope and the reliability data is current (Test 1); verify the Graph resource path against current Microsoft documentation.

Test 4 — Bad app vs. bad rollout is separated correctly

Procedure: For the app from Test 3, inspect the **version** breakdown in Application reliability. **Pass:** You can state which it is with evidence — **bad version/rollout** (one version bad, prior version clean, spike began right after that version deployed) **or bad app** (all versions crash across a broad, mixed population) — and you have identified the correct owner (deployment owner vs. app owner). **Fail:** The two cannot be distinguished → widen the observation window, confirm the version dimension is available, and re-read before routing (routing the wrong owner wastes the loop).

Test 5 — A restart trend is correctly attributed

Procedure: In the **device reliability / restart** report, examine the unexpected-restart trend and identify a rise (fleet-wide or model-scoped). Line its start time up against your change calendar. **Pass:** The rise is attributed to a plausible cause with evidence — **fleet-wide after an OS update** → update owner; **single hardware model** → driver/firmware owner — including the cohort and the start time. **Fail:** The spike can't be attributed → check whether the cohort is too small (high variance), widen the window, and confirm restart data is current before acting.

Test 6 — An anomaly is correctly interpreted (true signal)

Procedure: With anomaly detection available, open the **anomaly** view and pick an anomaly. Read the metric, the cohort, and the start time; cross-check it against the underlying report (Test 3 or Test 5). **Pass:** The anomaly corresponds to a **real movement** in application reliability or restart analytics, correlates with a change event, affects a meaningful cohort, and **persists** across a data refresh — i.e. it is a **true signal** worth triaging, and you routed it (Test 8). **Fail:** The "anomaly" has no corresponding movement in the underlying data → treat as suspect; see Test 7.

Test 7 — A false signal is correctly rejected

Procedure: For any anomaly that looks transient (tiny cohort, coincides with a reporting gap, no underlying movement), watch it across the next data refresh. **Pass:** The transient signal **fades** on refresh and is

correctly **not** actioned — no remediation effort spent, noted as a false lead. You can articulate *why* it was false (variance / reporting gap / no underlying movement). **Fail:** A false signal was escalated into a remediation → tighten the "confirm it's real" step (persistence + underlying-movement cross-check) before routing anomalies.

Test 8 — Triage routes to the right owner and closes the loop

Procedure: For a confirmed signal, create the handoff item `Reliability Handoff – <App|Driver|Update> – <Version>`, route it to the owner, and — where scriptable — hand the fix to the **automated-fix companion** (Proactive Remediations at Scale). After the fix, re-read the report and re-run `ENDPTREL-Test-Signal.ps1`. **Pass:** The signal routed to the correct owner with evidence; where scriptable, the fix was handed to the companion (not fixed by hand); and after the fix the signal **recovers** (crashes/hangs fall, restart trend flattens, or the anomaly clears) — `ENDPTREL-Test-Signal.ps1` now returns NOT DETECTED at problem levels. **Fail:** The signal doesn't recover → re-open the handoff, confirm the fix reached the affected cohort, and re-verify.

Test 9 — The review cadence and export work end to end

Procedure: Run `ENDPTREL-Report.ps1` and confirm it writes `Reliability Review – <YYYY-MM>.csv` with Application, Device, and Anomaly sections. Confirm the `Reliability – Review Scope` group exists (create it with `ENDPTREL-Set-ReviewScope.ps1 -Commit` if needed). **Pass:** The CSV is written with the three sections and the review scope group exists; the review can run on a schedule against it. **Fail:** Export empty or the group missing → resolve data availability (Test 1) and create the scope group before relying on the cadence.

Acceptance gate

Proceed from **Pilot** to **Expand** only when Tests 1–8 pass (Test 9 confirms the cadence is operable): reliability signals are populated, a known bad app and a restart trend are **correctly identified**, an anomaly is **correctly interpreted vs. a false signal**, and at least one confirmed signal has been **routed and verified to recover**.

6. Troubleshooting — Anomaly Detection & Application Reliability with Endpoint Analytics

Work top-down: confirm **reliability data is flowing** (parent solution), then **read the right signal**, then **classify** (bad app vs. bad rollout vs. environmental), then **route and verify**. Most problems are one of: data not populated yet, a false anomaly treated as real, or a crash that can't be pinned to a version because the version dimension isn't being read.

> ***Feature, licensing, and navigation specifics — verify against current Microsoft documentation.*** *Report names, blade paths, anomaly-detection availability, data latency, and minimum device populations change over time and can differ by cloud (Commercial vs. GCC / GCC High / DoD). Confirm the specifics for your own tenant rather than trusting a snapshot.*

Reliability data is missing or delayed

- **Endpoint Analytics not fully enabled / data collection not deployed.** This is the **parent**

solution's job. Confirm Endpoint Analytics is on and the data-collection policy is deployed and assigned to your devices; finish the parent solution before triaging here.

- **Reporting window hasn't filled.** Reliability signals are aggregate and delayed — a new tenant, a

newly onboarded scope, or a recent policy change needs time to populate. *(Exact latency — verify against current Microsoft documentation.)* Wait a full reporting window and re-check.

- **Too few devices in scope.** Aggregate signals need a meaningful population; a handful of devices

looks empty or noisy. Broaden the data-collection scope (parent solution) and re-check with `ENDPTREL-Discovery.ps1`.

- **Licensing/entitlement.** If Endpoint Analytics reports won't render at all, confirm the licensing

that includes Endpoint Analytics for your devices *(verify against current Microsoft documentation)*.

- **Graph resource path moved.** `ENDPTREL-Discovery.ps1` reads the `userExperienceAnalytics*` /beta

resources; if it reports "unavailable," the path may have changed — read the reports in the Intune admin center and verify the current Graph resource against Microsoft documentation.

The anomaly view is empty or unavailable

- **Anomaly detection not available in this cloud/tenant.** It may be preview, or absent in your cloud

(GCC / GCC High / DoD). *(Verify availability against current Microsoft documentation.)* Proceed with the application- and device-reliability portions, which don't require it, and adopt anomaly detection when available.

- **Below the minimum population/latency for anomalies.** Anomaly detection needs enough devices and

history to establish a baseline. Let the scope and window fill.

False anomalies (a signal that isn't real)

- **Transient / tiny-cohort variance.** A small cohort with high variance throws short-lived anomalies.

Watch it across the next data refresh — a **false** signal **fades**; a **true** signal **persists** and has corresponding movement in the underlying report.

- **No underlying movement.** If an anomaly has no matching rise in Application reliability or restart

analytics, treat it as suspect — don't route a remediation off the alert alone.

- **Reporting gap masquerading as a change.** A gap or late-reporting batch can look like a shift.

Confirm data continuity before attributing the anomaly to a real change event.

- **Rule of thumb:** confirm **persistence + underlying movement + a plausible change event** before

spending remediation effort. (Testing guide, Tests 6–7.)

Can't attribute a crash to a specific app version

• **Version dimension not being read.** In Application reliability, drill into the **version** breakdown — a crash is only routable once you know whether one version is bad (bad rollout) or all are (bad app). If the version dimension isn't exposed for that app, note it and route on the app-level signal.

• **Signal too fresh.** A brand-new version needs enough installs/time to produce a stable per-version signal. Widen the window.

• **App identity ambiguity.** Similarly-named apps can split a signal. Confirm you're reading the right app (publisher/identifier) with `ENDPTREL-Test-Signal.ps1 -AppNameLike '<app>'`.

Restart cause is unclear

• **Cohort not scoped.** Decide first whether the unexpected-restart rise is **fleet-wide** (suspect an OS update) or **single-model** (suspect a driver/firmware) — the total alone won't tell you. Filter the restart report by model.

• **No change-calendar correlation.** Line the trend's start time up against your update rings, driver pushes, and config changes. No correlation usually means it's environmental/noise or the window is too short.

• **Planned vs. unexpected mixed together.** Make sure you're trending **unexpected** restarts (stop-errors), not planned reboots (updates, user-initiated), which will otherwise mask the real signal.

The fix shipped but the signal didn't recover

• **Fix didn't reach the affected cohort.** Confirm the app update/rollback, driver rollback, update-ring change, or the automated-fix companion's remediation actually targeted the population showing the signal (align with `Reliability – Review Scope` or a tighter cohort).

• **Reporting lag after the fix.** Recovery is also delayed — give it a reporting window, then re-read and re-run `ENDPTREL-Test-Signal.ps1` (expect NOT DETECTED at problem levels).

• **Wrong owner / wrong root cause.** If it truly won't recover, revisit the bad-app-vs-bad-rollout classification — you may have routed the wrong owner.

Licensing & scope

• **Reports won't load / feature absent.** Verify the Endpoint Analytics entitlement and that anomaly detection is available for your cloud *(verify against current Microsoft documentation)*.

• **Review scope drifts from data collection.** Keep `Reliability – Review Scope` aligned with the population Endpoint Analytics is collecting, or you'll review a different set than what's reporting.

• **Reader vs. writer permissions.** Reading the reports needs read access (R6); creating `Reliability – Review Scope` needs `Group.ReadWrite.All` (R7). A "can read but can't create the group" symptom is a permissions split, not a data problem.

Scripts

• `ENDPTREL-Discovery.ps1` *(read-only)* — first stop when data looks missing: reports whether the reliability signals return data, the scoped device count, and the top unreliable apps/devices.

• `ENDPTREL-Validate.ps1` *(read-only)* — the PASS/FAIL gate; an anomaly-detection **WARN** is acceptable (unavailable/preview), a data **FAIL** is not.

• `ENDPTREL-Report.ps1` *(read-only)* — if the export is empty, the underlying data is missing (see "Reliability data is missing or delayed"), not the script.

• `ENDPTREL-Test-Signal.ps1` *(read-only)* — use it both to confirm a bad app's signal is attributable and, after a fix, to prove it receded (DETECTED → NOT DETECTED).

• `ENDPTREL-Set-ReviewScope.ps1` *(state-changer — defaults to `-WhatIf`)* — if "nothing happened," that's by design: it previews only until re-run with `-Commit`. Creating the group needs `Group.ReadWrite.All`.

• **General:** all reads use the `userExperienceAnalytics*` /beta Graph resources, which **change over time and vary by cloud** — a script reporting "unavailable / path moved" is a signal to verify the current resource against Microsoft documentation, not necessarily a tenant problem. Re-run the relevant script after each change to confirm the gate.

7. Best Practices — Anomaly Detection & Application Reliability with Endpoint Analytics

Field-proven guidance for running the reliability side of Endpoint Analytics as a durable practice.

Read the signals the right way

• **Never act on a single data point.** Reliability signals are aggregate and delayed — confirm a trend across a data refresh before you route a fix. *(Exact data latency — verify against current Microsoft documentation.)*

• **Always check the version dimension first** for an app problem. One bad version with a clean prior version is a **bad rollout**; all versions crashing across a broad, mixed population is a **bad app**. The version breakdown is the fastest way to route the right owner.

• **Scope restart trends before blaming a cause.** A model-scoped restart spike is a driver/firmware lead; a fleet-wide spike right after an OS update is an update lead. Read the cohort, not just the total.

• **Treat an anomaly as a lead, not a verdict.** Read the metric, the cohort, and the start time, then cross-check it against the underlying report. Confirm it's real (persists + has underlying movement) before spending remediation effort.

Separate a bad app from a bad rollout — every time

• **Common factor is the app** → **bad app.** Broad, mixed population, all versions. Fix/update/roll back the app; owner is the app owner.

• **Common factor is the change event** → **bad rollout.** Concentrated in the population that got a new version/config, started right after. Pause/stage/roll back the deployment; owner is the deployment owner.

• **When in doubt, widen the window and narrow the cohort** — more time confirms the trend, a tighter cohort confirms the common factor.

Route to the right owner and close the loop

• **One handoff, one label.** Use `Reliability Handoff – <App|Driver|Update> – <Version>` so every fix is traceable from signal to owner to recovery.

• **Attach the evidence.** The report row/export (`Reliability Review – <YYYY-MM>.csv`), the affected cohort, and the start time — owners fix faster with the specifics.

• **Verify recovery before closing.** The loop is closed only when the **same** signal recovers, not when the fix ships. Re-read the report and re-run `ENDPTREL-Test-Signal.ps1`.

Hand scriptable fixes to the automated-fix companion

• **Don't fix fleet-wide problems by hand.** Where the remediation is scriptable — clear a bad cache, reset a service, roll back a driver, repair a config — hand it to **"Proactive Remediations at Scale"** (the automated-fix companion) with the exact detect/remediate conditions.

• **This product decides *what* and *who*; the companion decides *how* at scale.** Keep that division — it's what makes the two solutions compose instead of overlap.

• **Target the fix at the affected cohort**, not the whole fleet, unless the problem is genuinely fleet-wide.

Lean on the parent for the data, not this product

• **Data problems are the parent's job.** Empty/thin reliability data, scope, and licensing belong to the **parent** Endpoint Analytics solution — don't try to fix data collection from inside the reliability practice; fix it upstream and return.

• **Keep the review scope aligned with data collection.** `Reliability – Review Scope` should mirror the population Endpoint Analytics is collecting, so you review exactly what's reporting.

Make it a cadence, and measure it

- **Run a standing reliability review** (`Reliability Review – Weekly` to start, monthly once steady)

against `Reliability – Review Scope`, working the read-triage-route-verify loop every time.

- **Keep the monthly export** (`Reliability Review – <YYYY-MM>.csv`) as the trend record.
- **Track corrections-per-review over time.** A downward trend means you're catching regressions early

and preventing ticket waves — the whole point of the practice.

- **Rehearse on the pilot first.** Prove the loop on `Reliability – Pilot` before widening to the

fleet.

Respect national-cloud differences

- **Verify Government parity.** For GCC / GCC High / DoD tenants, portal URLs and feature availability

(including whether anomaly detection is present) can differ — verify against Microsoft's US-sovereign-cloud documentation rather than assuming Commercial behavior.

8. Deliverables Summary — Anomaly Detection & Application Reliability with Endpoint Analytics

Everything included in this solution package, and what each piece is for.

Documentation (this guide)

- **Executive Summary** — the business case and outcome in one page.
- **Solution Overview** — the three reliability signals (application reliability, device reliability /

restart analytics, anomaly detection), how to read each, the bad-app-vs-bad-rollout distinction, the triage-and-close-the-loop practice, and how this product relates to the **parent** Endpoint Analytics solution and the **automated-fix companion** (Proactive Remediations at Scale).

- **Requirements Guide** — every prerequisite (Endpoint Analytics enabled and reliability data flowing

via the parent, licensing/entitlement, anomaly-detection availability, sufficient device population, device/data scope, reader access, group-creation rights) with how to **verify** and how to **implement if missing**, plus the category-level Global Requirements. Feature/licensing/navigation specifics are flagged **verify against current Microsoft documentation**.

- **Implementation Guide** — exact, ordered steps across six phases: confirm reliability data is

flowing, read application reliability and separate bad-app from bad-rollout, read device reliability / restart analytics, use anomaly detection to catch a regression, triage and route (handing scriptable fixes to the automated-fix companion), then establish the review cadence and expand. Every object is named and reused verbatim.

• **Testing & Validation** — nine tests with explicit pass/fail gates and an acceptance gate, proving the signals are populated, a known bad app and a restart trend are correctly identified, an anomaly is correctly interpreted vs. a false signal, and a confirmed signal is routed and verified to recover.

• **Troubleshooting** — top-down diagnosis for missing/delayed reliability data, false anomalies, crashes that can't be attributed to a version, unclear restart causes, and licensing/scope, plus a scripts section.

• **Best Practices** — how to read the signals, separate bad app from bad rollout, route and close the loop, hand scriptable fixes to the companion, and run the cadence as a measured habit.

PowerShell scripts (Microsoft Graph) — `ENDPTREL-Scripts.zip`

All scripts are tenant-side (run from an admin workstation with the Microsoft Graph PowerShell SDK); each header states **Purpose · How to run · Graph scopes · Expected results**. Four are **read-only**; one is a state-changer that **defaults to** `-WhatIf`.

1. `ENDPTREL-Discovery.ps1` *(read-only)* — reliability-data availability, the scoped device count, and the top unreliable apps/devices.

1. `ENDPTREL-Validate.ps1` *(read-only)* — PASS/FAIL readiness gate: reliability signals present for the pilot, anomaly detection active.

1. `ENDPTREL-Report.ps1` *(read-only)* — exports the application + device reliability + anomaly summary to `Reliability Review – <YYYY-MM>.csv` for the review cadence.

1. `ENDPTREL-Test-Signal.ps1` *(read-only)* — confirms a known reliability signal is visible and attributable to an app/version (and, after a fix, that it has receded).

1. `ENDPTREL-Set-ReviewScope.ps1` *(state-changer — defaults to `-WhatIf`)* — creates/assigns the `Reliability – Review Scope` group for the cadence. Previews only until re-run with `-Commit`.

> *Reliability + anomaly data is read from the `userExperienceAnalytics*` Microsoft Graph resources, which evolve over time and can vary by cloud — the scripts fail soft and flag **verify against current Microsoft documentation** rather than erroring if a resource path has moved.*

Hero image

• `ENDPTREL-hero.png` — the product image (1200×1200) used on the marketplace listing.

How it's delivered

A **Word** document and **PDF** (cover, table of contents, page numbers, and a scripts appendix), plus all five PowerShell scripts in a single **ZIP** — yours to keep and reuse across **unlimited devices**.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

ENDPTREL-Discovery.ps1

Purpose: Confirms the reliability side of Endpoint Analytics is producing data this solution can act on: that application-reliability and device/restart signals are present, reports the scoped device count, and lists the top unreliable apps and devices it can see. This is the "is the data flowing?" check that Phase 1 of the Implementation guide calls for. It changes NOTHING.

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once .\ENDPTREL-Discovery.ps1 .\ENDPTREL-Discovery.ps1 -Top 15 Reads only — makes no changes.

ENDPTREL-Report.ps1

Purpose: Produces the monthly reliability record the review cadence keeps: one CSV combining the top unreliable applications (crash/hang by app/version), the least-reliable devices, and any active anomalies. This is the 'Reliability Review – <YYYY-MM>.csv' artifact referenced throughout the Implementation and Best Practices guides. Read-only — it only reads and writes a local CSV.

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once .\ENDPTREL-Report.ps1 .\ENDPTREL-Report.ps1 -Top 25 -OutDir 'C:\Reliability' Reads Graph and writes a local CSV — makes no tenant changes.

ENDPTREL-Set-ReviewScope.ps1

Purpose: Stands up the ongoing review population for the reliability review cadence: an Entra security group named 'Reliability – Review Scope'. This is the ONLY state-changing script in this solution, and it DEFAULTS TO -WhatIf (preview only) — it will NOT create or modify anything until you explicitly re-run with -Commit (or -WhatIf:$false). By default it creates an ASSIGNED security group (you add the in-scope devices), aligned with the population Endpoint Analytics is collecting. Optionally, with -Dynamic, it creates a dynamic device group from a rule you supply. Idempotent — if the group already exists it reports and does nothing.

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once .\ENDPTREL-Set-ReviewScope.ps1 # PREVIEW ONLY (-WhatIf default) — no change .\ENDPTREL-Set-ReviewScope.ps1 -Commit # actually create/assign the group .\ENDPTREL-Set-ReviewScope.ps1 -Dynamic -MembershipRule '(device.deviceOSType -eq "Windows")' -Commit

ENDPTREL-Test-Signal.ps1

Purpose: Given the name of an application you know is failing (a seeded/known bad app, or one flagged in the Application reliability report), confirms the signal is actually VISIBLE in Endpoint Analytics and ATTRIBUTABLE to that app (and its version where exposed). This is the read-only check the Testing guide uses to prove a true signal is being detected — and, after a fix, that the signal has receded. It changes NOTHING.

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once .\ENDPTREL-Test-Signal.ps1 -AppNameLike 'Contoso LOB' .\ENDPTREL-Test-Signal.ps1 -AppNameLike 'Contoso LOB'

-MinCrashCount 5 Reads only — makes no changes.

ENDPTREL-Validate.ps1

Purpose: Confirms the pieces this solution needs are in place for a pilot: reliability signals are present for the scoped devices, the pilot group exists, and anomaly detection appears active. Emits a PASS/FAIL line per check and an overall verdict — the acceptance gate before you widen the reliability review from pilot to fleet. Read-only.

How to run: Install-Module Microsoft.Graph -Scope CurrentUser # once .\ENDPTREL-Validate.ps1 .\ENDPTREL-Validate.ps1 -PilotGroupName 'Reliability – Pilot' Reads only — makes no changes.